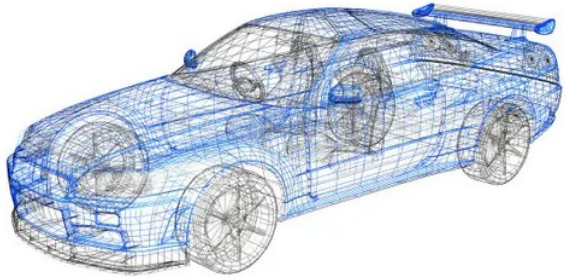
O Conceito de Classe e Objeto



Flávio Gomes da Silva Lisboa
www.fgsl.eti.br

Bento Quirino
Escola Técnica Estadual

Complexidade

Uma estrela moribunda à beira do colapso, uma criança aprendendo a ler, glóbulos brancos correndo para atacar um vírus: esses são apenas alguns exemplos do mundo físico que carregam uma complexidade verdadeiramente aterradora. O software também pode envolver elementos de grande complexidade; no entanto, a complexidade encontrada aqui é de um tipo fundamentalmente diferente.

(CARDACCI; BOOCH, 2013, p. 21, nossa tradução)

Programação Orientada a Objetos

A programação orientada a objetos, ou POO (do inglês Object-Oriented Programming), é o paradigma de programação dominante atualmente, tendo substituído as técnicas de programação “estruturada” ou procedural desenvolvidas na década de 1970. Como o Java é orientado a objetos, é necessário ter familiaridade com POO para se tornar produtivo com Java.

(HORSTMANN, 2019, p. 178, nossa tradução)

Classe e objeto

Uma **classe** é similar em termos do conceito aos **desenhos** de engenharia de um carro, que armazenam o projeto de um pedal de acelerador, volante etc.

[...]

Assim como alguém tem de **fabricar** um carro a partir dos desenhos de engenharia antes que possa realmente dirigi-lo, você deve **construir** um objeto de uma classe antes que um programa possa executar as tarefas que os métodos da classe definem. O processo para fazer isso é chamado **instanciação**. **Um objeto é então referido como uma instância da sua classe**.

(DEITEL e DEITEL, 2017, p. 9)

Classe e objeto

Um conceito fundamental no design orientado a objetos é a diferenciação entre objetos e classes. Um objeto é qualquer entidade específica que existe no seu programa em tempo de execução. Uma **classe** é a entidade **estática** que você vê na listagem do programa. Um **objeto** é a entidade **dinâmica** com valores e atributos específicos que você vê quando executa o programa.

(MCCONNELL, 2004, p. 86, nossa tradução)

Classe e objeto

Por exemplo, você poderia declarar uma classe ‘Pessoa’ que tivesse os atributos nome, idade, sexo e assim por diante. Em tempo de execução, você teria os objetos ‘Nancy’, ‘Hank’, ‘Diane’, ‘Tony’ e assim por diante — ou seja, instâncias específicas da classe. Se você estiver familiarizado com termos de banco de dados, é o mesmo que a distinção entre “esquema” e “instância”. Você pode pensar na classe como o cortador [ou forma] de biscoitos e no objeto como o biscoito.

(MCCONNELL, 2004, p. 86, nossa tradução)

Classe e objeto

Objetos do mundo real não são o único tipo de objeto de interesse no desenvolvimento de software. Outros tipos importantes de objetos são invenções do processo de projeto, cujas colaborações com outros objetos semelhantes servem como mecanismos para executar algum comportamento de nível superior.

(CARDACCI; BOOCH, 2013, p. 106, nossa tradução)

Programação Estruturada X Programação Orientada a Objetos

A programação estruturada tradicional consiste em projetar um conjunto de procedimentos (ou algoritmos) para resolver um problema. Uma vez definidos os procedimentos, o próximo passo tradicional era encontrar maneiras adequadas de armazenar os dados. É por isso que o criador da linguagem Pascal, Niklaus Wirth, intitulou seu famoso livro sobre programação de Algoritmos + Estruturas de Dados = Programas (Prentice Hall, 1975).

(HORSTMANN, 2019, p. 179, nossa tradução)

Programação Estruturada X Programação Orientada a Objetos

Observe que, no título de Wirth, os algoritmos vêm primeiro e as estruturas de dados em segundo lugar. Isso reflete a maneira como os programadores trabalhavam naquela época. Primeiro, eles decidiam os procedimentos para manipular os dados; depois, decidiam qual estrutura impor aos dados para facilitar as manipulações. A Programação Orientada a Objetos (POO) inverte essa ordem: **coloca os dados em primeiro lugar** e, em seguida, analisa os algoritmos para operar sobre os dados.

(HORSTMANN, 2019, p. 179, nossa tradução)

Quando usar Programação Orientada a Objetos

Para problemas pequenos, a decomposição em procedimentos funciona muito bem. Mas objetos são mais apropriados para problemas maiores.

(HORSTMANN, 2019, p. 179, nossa tradução)

Referências

- CARDACCI, Darío. BOOCH, Grady. **Orientación a objetos**: teoría y práctica. Buenos Aires: Pearson Education, 2013.
- DEITEL, Paul. DEITEL, Harvey. **Java**: como programar. Tradução: Edson Furmankiewicz. Revisão técnica: Fabio Lucchini. São Paulo: Pearson Education do Brasil, 2017.
- HORSTMANN, Cay S. **Core Java**. Volume I – Fundamentals. 11. ed. Boston: Pearson, 2019.
- MCCONNELL, Steve. **Code Complete**. 2. ed. Redmond: Microsoft Press, 2004.